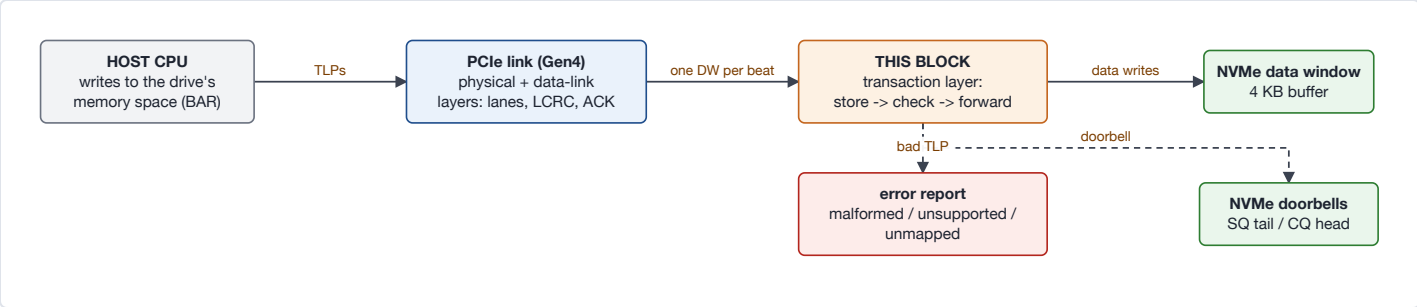
YOUR UVM REPO · PCIE_NVME

PCIe Gen4 TLP to NVMe Hand-off

What every file does · where it sits in an SSD controller · PCIe layers and the Memory Write TLP bit by bit · NVMe doorbells · worked TLPs from wire to NVMe · design and UVM flow step by step · 10 constraints · 10 assertions · results

1. What this block does, and why

An NVMe SSD talks to the computer over **PCIe**. Everything the host sends to the drive — commands, data, "I have new work for you" signals — travels as **packets** called **TLPs** (Transaction Layer Packets). Something inside the drive must **open each packet, check that it is legal, decide where it goes, and hand it to the NVMe controller**. That is this block.



PCIe in three layers (the same for Gen1 to Gen5):

Layer	Job	In this design?
Physical	lanes, encoding (Gen4: 16 GT/s per lane, 128b/130b)	no
Data Link	sequence numbers, **LCRC**, ACK/NAK replay — makes the link reliable	no (assumed done)
Transaction	builds and **checks TLPs**, decodes addresses, flow control	**yes**

Gen4 note: Gen4 doubled the speed of Gen3 and allows 10-bit tags, but the **TLP format is the same** as Gen1–Gen5, so this block is "Gen4" by following that format (tag bits T9/T8 are 0 here).

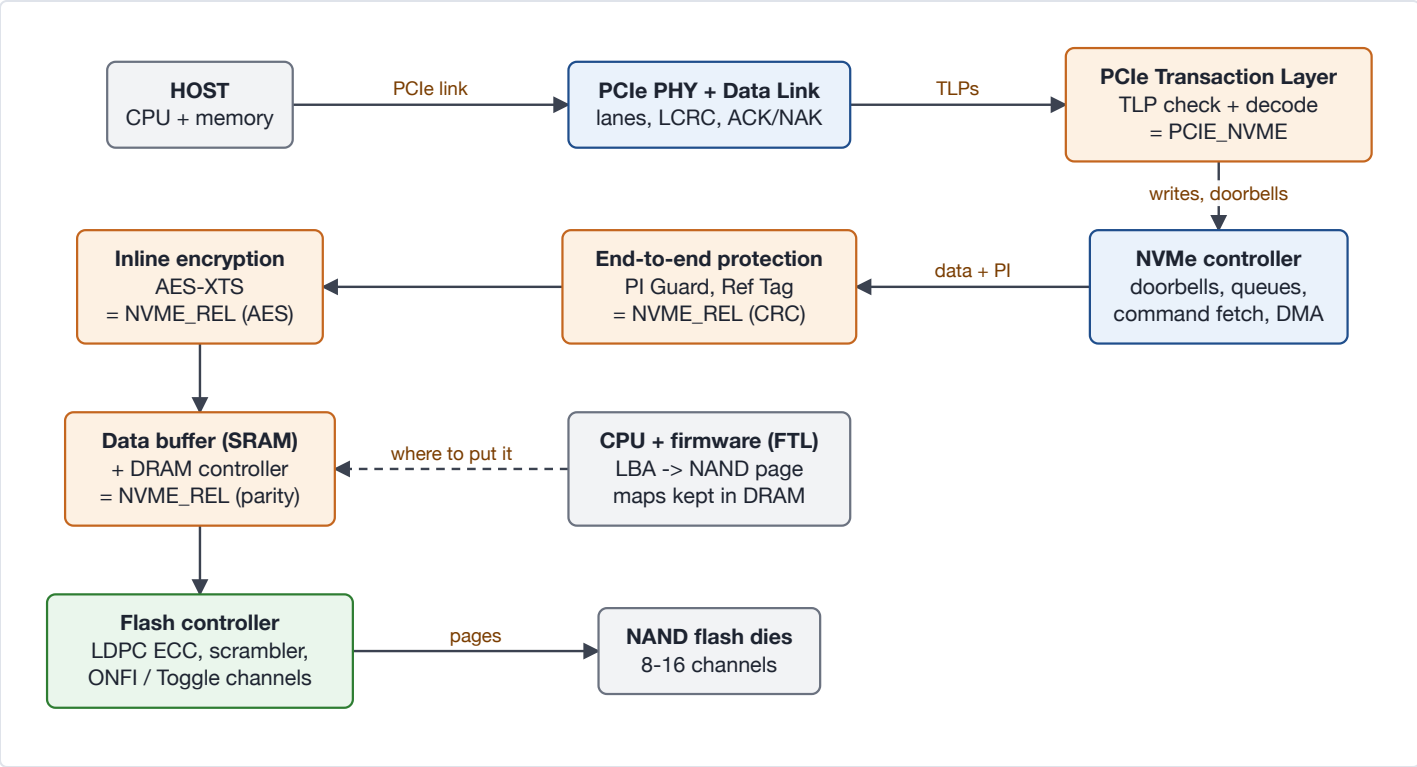
Where this fits in NVMe: the host rings an NVMe **doorbell** by writing a register in the drive's BAR (a Memory Write TLP) — "Submission Queue 1 now has 5 entries". For a large data transfer the drive usually **reads** host memory itself (Memory Read → Completion with Data). This block handles the host-to-drive **Memory Writes**: doorbell writes, and data written

into a 4 KB device buffer window (like NVMe's Controller Memory Buffer). Memory Reads are reported as "unsupported" here, to keep it simple.

Store, check, forward: the block keeps the whole TLP in a small buffer **before** forwarding anything. A broken TLP must be dropped completely — forwarding half of it and then discovering it was malformed would corrupt the NVMe buffer.

2. Where this IP sits inside an SSD controller

An SSD controller is one chip with a **front end** (talks to the computer), a **middle** (moves and protects data, runs the firmware), and a **back end** (talks to the NAND flash). A write follows the arrows from the host down to the NAND; a read travels the same path backwards.



Part of the controller	What it does	Which of your blocks lives here
PCIe PHY + Data Link Layer	turns bits on the lanes into reliable packets: encoding, LCRC on every TLP, ACK/NAK replay	— (assumed done before your block)
PCIe Transaction Layer	checks each TLP (type, length, 4 KB rule), decodes the address against the BARs, reports Unsupported Request / Malformed TLP	**PCIE_NVME** — exactly this job
NVMe controller (front end)	doorbell registers, submission/completion queues, fetches commands, runs the **DMA** that moves data between host memory and the drive	**PCIE_NVME** hands it doorbell writes and data-window writes
End-to-end data protection	generates or checks the **Protection Information** (CRC Guard, App Tag, Ref Tag) at the host interface, and checks it again just before the media — so corruption anywhere inside the drive is caught	**NVME_REL** — CRC-16 / CRC-32C Guard, Ref Tag, App Tag escape

Part of the controller	What it does	Which of your blocks lives here
Inline encryption engine	encrypts every block on the way to NAND and decrypts on the way back (self-encrypting drive); real drives use **AES-XTS** with the LBA as the "tweak" so equal data at different LBAs looks different	**NVME_REL** — the AES-128 core (one block, no XTS tweak, to keep it simple)
Data buffer + DRAM (memory) controller	SRAM staging buffers and the DRAM that holds data in flight and the firmware's mapping tables; protected by **parity** (SRAM) or **ECC** (DRAM) because a bit flip here would silently corrupt user data or the map	**NVME_REL** — the byte parity
CPU cores + firmware (FTL)	the Flash Translation Layer decides which NAND page holds each LBA, does wear levelling and garbage collection	— (software)
Flash controller (back end)	strong **ECC** (LDPC or BCH) per NAND page, a **scrambler** so data looks random to the cells, read-retry, and the channel interface to the NAND dies	— (not in these blocks; the next level of protection)
NAND flash	the storage cells, many dies on several channels	—

Why so many layers of protection? Each one guards a different stretch of the path, like checkpoints on a road:

Protection	Guards	Strength
LCRC (PCIe Data Link)	the PCIe wire only	re-sends a bad packet
PI Guard CRC (NVME_REL)	host → whole controller → media, **end to end**	detects; the command fails instead of returning wrong data
Parity / DRAM ECC (NVME_REL parity)	one memory inside the controller	parity detects 1 bit; ECC corrects 1, detects 2
LDPC / BCH ECC (flash controller)	the NAND cells, which really do wear out and flip bits	corrects many bits per page
AES-XTS (NVME_REL AES)	not errors — **privacy** of the stored data	—

One write, from host to NAND, in order: host rings the SQ doorbell (**PCIE_NVME** decodes the TLP → doorbell) → NVMe controller fetches the command and DMAs the data in (more TLPs through **PCIE_NVME**) → **PI** is checked/generated (**NVME_REL** CRC) → data parked in the buffer (**parity**) → **encrypted** (**NVME_REL** AES) → firmware picks a NAND page → flash controller adds LDPC ECC and scrambles → programmed into NAND. A **read** runs the same path backwards: LDPC corrects the page → decrypt → PI checked again → DMA to the host.

3. The packet: a Memory Write TLP, bit by bit (PCIe Base Specification)

A TLP is a **header** of 3 or 4 DWORDs (a DWORD, "DW", is 32 bits) followed by the **payload**. On this interface one DW moves per clock beat: `tlp_sop` marks the first beat, `tlp_eop` the last.

DW0 — what kind of TLP, and how long:

Bits	Field	Value for a Memory Write
31:29	**Fmt**	`010` = 3-DW header **with data**; `011` = 4-DW header with data

Bits	Field	Value for a Memory Write
28:24	**Type**	`00000` = Memory Request
23	T9 (tag bit 9, Gen4 10-bit tags)	0
22:20	TC (traffic class)	0
19	T8 (tag bit 8)	0
18, 17, 16	Attr[2], LN, TH	0
15	TD (a TLP digest / ECRC follows)	0
14	EP (poisoned data)	0
13:12	Attr (relaxed ordering, no-snoop)	0
11:10	AT (address translation)	0
9:0	**Length**	payload size in DWs; **0 means 1024**

DW1 — who sent it, and which bytes count:

Bits	Field	Meaning
31:16	**Requester ID**	bus / device / function of the sender
15:8	**Tag**	the sender's number for this request
7:4	**Last DW BE**	which bytes of the **last** payload DW are valid
3:0	**First DW BE**	which bytes of the **first** payload DW are valid

DW2 (and DW3) — the address: 3-DW header: DW2 = address[31:2] (bits 1:0 are 0: DW aligned). 4-DW header: DW2 = address[63:32], DW3 = address[31:2].

Fmt / Type you should recognise:

TLP	Fmt	Type	Data?
Memory Read (MRd)	000 / 001	00000	no — **non-posted**: the completer answers with a CplD
Memory Write (MWr)	010 / 011	00000	yes — **posted** (no answer)
Completion with Data (CplD)	010	01010	yes
Completion without data (Cpl)	000	01010	no

The spec rules this design checks or obeys:

Rule	Source	Here
Payload size must equal Length	Transaction layer: otherwise **malformed**	DUT checks → error code 1
A request must not cross a **4 KB** address boundary	Memory requests	DUT checks → code 1; constraint keeps good TLPs inside
4-DW header only for addresses ≥ 4 GB	Requester rule	constraint 2
Length 1 ⇒ Last DW BE = 0000	Byte-enable rules	constraint 8

Rule	Source	Here
Max Payload Size: every device supports 128 bytes	Device Control register	Length ≤ 32 DW (constraint 6)
Only supported requests are accepted	Unsupported Request (UR)	anything but MWr → code 2

NVMe doorbells (NVMe base specification, controller registers): Submission Queue y Tail Doorbell at **1000h + (2y × (4 << CAP.DSTRD))**, Completion Queue y Head Doorbell at **1000h + ((2y + 1) × (4 << CAP.DSTRD))**. With DSTRD = 0 the stride is 4 bytes:

Offset in BAR0	Register
1000h	SQ0 Tail Doorbell (admin queue)
1004h	CQ0 Head Doorbell
1008h	SQ1 Tail Doorbell
100Ch	CQ1 Head Doorbell
… 101Ch	CQ3 Head Doorbell

This design's address map: BAR0 = 8000_0000h; doorbells at BAR0 + 1000h … 101Fh; data window at BAR0 + 2000h … 2FFFh (4 KB); anything else is **unmapped** (code 3).

4. Worked examples — one TLP from the wire to the NVMe side

Example 1: write 2 DWs into the data window. Requester 0100h, tag 05h, all bytes valid, address 8000_2010h, data `11223344`, `55667788`.

Beat	DW	Hex	How it's built
1 (sop)	DW0	`4000_0002`	Fmt `010` + Type `00000` = `0100 0000` = `40`; Length = 2
2	DW1	`0100_05FF`	Requester `0100`, Tag `05`, Last BE `F`, First BE `F`
3	DW2	`8000_2010`	address[31:2] with bits 1:0 = 00
4	data	`1122_3344`	
5 (eop)	data	`5566_7788`	

What the DUT does:

State	Decision
RECV	5 beats stored in `buffer[0..4]`
CHECK	Fmt[1] = 1 and Type = 0 → Memory Write ✓; payload 5 − 3 = 2 = Length ✓; 010h + 2 × 4 = 018h ≤ 4096 → no 4 KB crossing ✓; 8000_2010h is inside the window ✓
SEND beat 1	`nvme_addr` = 010h, `nvme_data` = `11223344`, `nvme_be` = F (First BE)

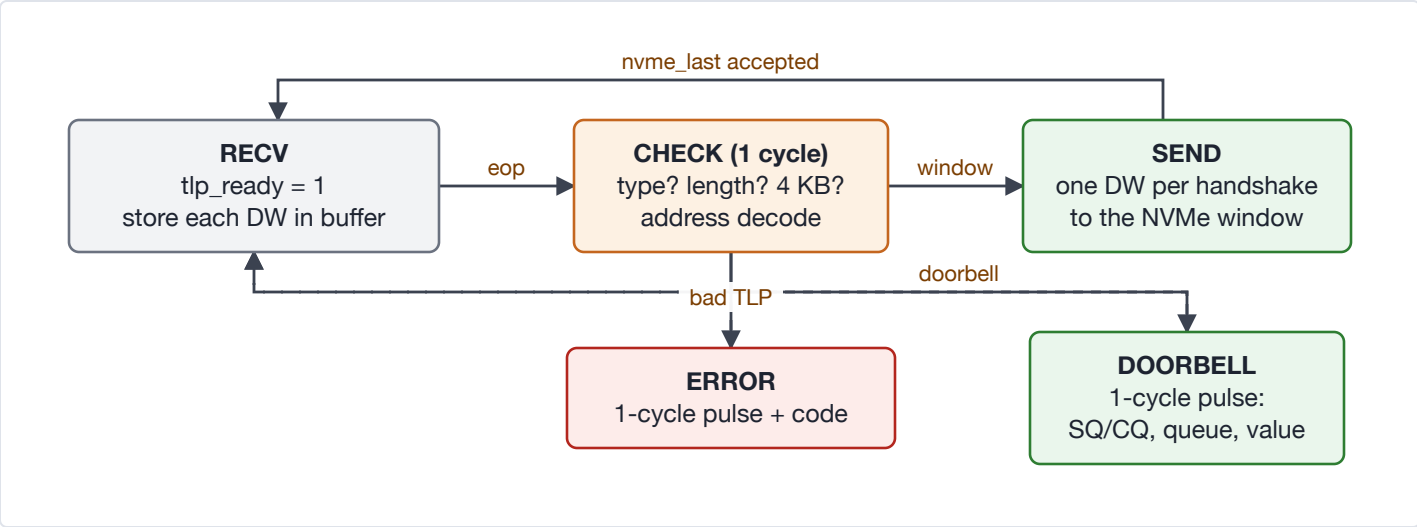
State	Decision
SEND beat 2	`nvme_addr` = 014h, `nvme_data` = `55667788`, `nvme_be` = F (Last BE), `nvme_last` = 1

Example 2: ring the SQ1 tail doorbell with value 5. DW0 `4000_0001` (Length 1), DW1 `0100_060F` (tag 06, Last BE 0 because Length = 1, First BE F), DW2 `8000_1008`, data `0000_0005`. CHECK: offset 1008h is in the doorbell range and Length = 1 → DOORBELL: `db_cq` = address bit 2 = 0 (SQ), `db_qid` = address bits 4:3 = 01 (queue 1), `db_value` = 5.

Example 3: three ways to be rejected.

TLP	Error
Fmt `000` (a Memory Read)	code 2, Unsupported Request
Length = 2 but 3 payload DWs	code 1, Malformed
Address 8000_4000h	code 3, Unmapped

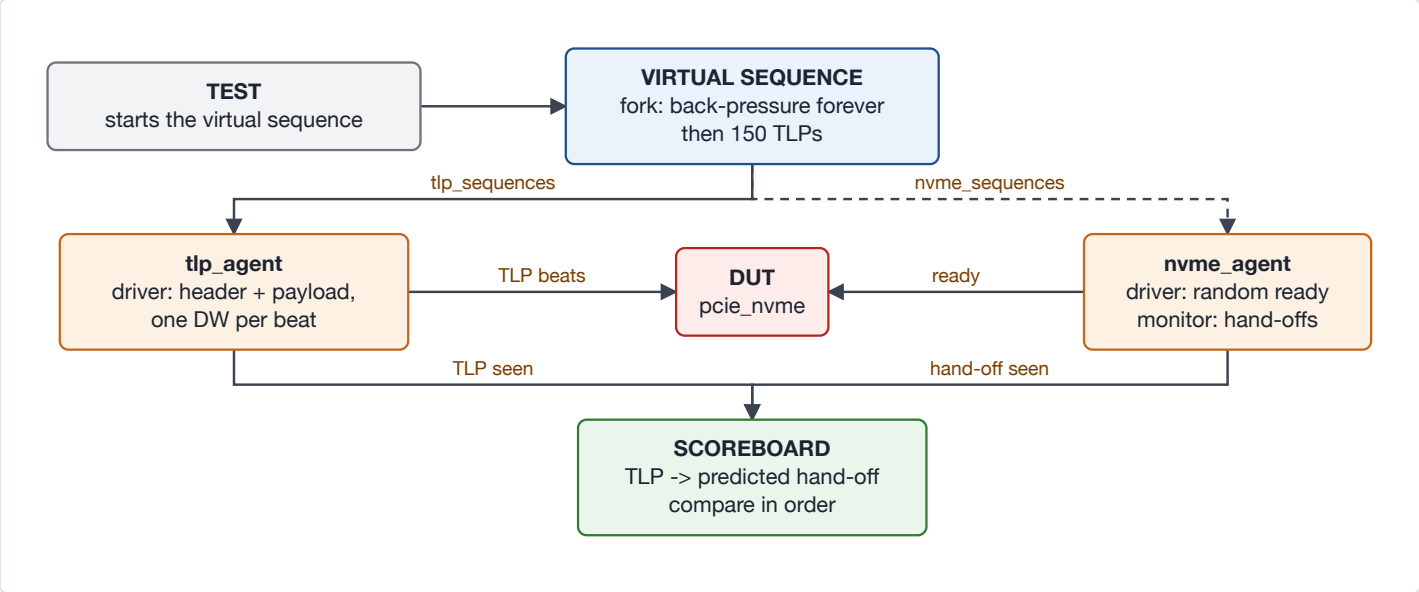
5. The design, step by step



Step	What `pcie_nvme.sv` does	Why
RECV	`tlp_ready` = 1; each beat goes to `buffer[count]`; `sop` restarts at 0; on `eop` → CHECK	keep the whole TLP before deciding
CHECK	read Fmt/Type/Length/BEs/address from the buffer; test in the order: supported? payload = Length? 4 KB? window? doorbell?	the order of the spec's checks: a malformed TLP is malformed whatever its address
SEND	for i = 0 … Length−1: `nvme_valid` with address = window offset + 4i, data = buffer[header + i], BE = First / F / Last; wait for `nvme_ready` each time	the NVMe side may be busy (back-pressure)
DOORBELL	address bit 2 → SQ or CQ; bits 4:3 → queue number; data[15:0] → new tail/head	the NVMe doorbell layout
ERROR	`err_valid` + code for one cycle; the TLP is dropped	malformed / unsupported / unmapped

While the block is in CHECK, SEND, DOORBELL or ERROR, `tlp_ready` is 0 — that's how back-pressure from the NVMe side reaches the PCIe side.

6. The UVM testbench, step by step



1. `tb_top` connects the DUT to two interfaces: `tlp_if` (PCIe side) and `nvme_if` (NVMe side), and puts both in `config_db`.
2. The **test** starts `pcie_nvme_vseq` on the **virtual sequencer**.
3. The virtual sequence `fork`s `nvme_sequences` on the NVMe agent (random `nvme_ready`, forever, in the background — `join_none`) and runs `tlp_sequences` (150 TLPs) on the PCIe agent.
4. Each `tlp_item` is randomized under the constraints; `header()` builds DW0–DW2/3 **exactly as in section 3**.
5. The **TLP driver** sends header then payload, one DW per beat, `sop` on the first, `eop` on the last; a beat is done only when `valid && ready` are seen at the same edge.
6. The **TLP monitor** collects beats from `sop` to `eop` and decodes the header again (independently of the driver) → `tlp_item` on its analysis port.
7. The **NVMe monitor** builds one event per hand-off: a whole data write (first beat … `nvme_last`), a doorbell, or an error.
8. The **scoreboard** turns each TLP into the hand-off the spec requires (same checks as section 5) and compares it with what the NVMe monitor saw, in order.
9. The 10 **assertions** watch both interfaces; **coverage** samples every TLP (Fmt, target, length, byte enables).
10. `pcie_nvme_err_test` overrides `tlp_item` with `tlp_err_item`: every TLP has exactly one problem.

7. Every file and what it does

File	What it does
`pcie_nvme.sv`	**DUT**: store the TLP, check it (type, length, 4 KB, address), forward data / doorbell / error

File	What it does
`tlp_if.sv`	PCIe-side interface + assertions 1–4 (hold, sop/eop with valid, framing, no X)
`nvme_if.sv`	NVMe-side interface + assertions 5–10 (hold, alignment, BE, doorbell pulse, error pulse, reset)
`testbench.sv`	Top: clock, reset, DUT, both interfaces → `config_db` , `run_test`
`pcie_nvme_pkg.sv`	Package + the address map (BAR, doorbell base, window base)
`pcie_nvme_agent_cfg.sv`	Active/passive, coverage on/off (shared by both agents)
`tlp_item.sv`	**The TLP**: Fmt, Type, Length, Requester ID, Tag, BEs, address, payload; `header()` packs DW0–DW3; constraints 1–9
`tlp_err_item.sv`	Error TLP: replaces `c_good` → exactly one of bad length / bad type / unmapped
`tlp_sequences.sv` , `tlp_sequencer.sv`	150 random TLPs; sequencer
`tlp_driver.sv`	Header + payload, one DW per beat, `sop` / `eop`
`tlp_monitor.sv`	Re-assembles TLPs and decodes the header per the spec
`tlp_agent.sv`	PCIe-side agent
`nvme_item.sv`	`nvme_ready` pattern for back-pressure — constraint 10
`nvme_evt.sv`	What the NVMe side received: write (address, DWs, BEs), doorbell, or error
`nvme_sequences.sv` , `nvme_sequencer.sv`	Forever back-pressure sequence; sequencer
`nvme_driver.sv`	Reactive side: only drives `nvme_ready`
`nvme_monitor.sv`	Builds one `nvme_evt` per hand-off
`nvme_agent.sv`	NVMe-side agent
`pcie_nvme_scoreboard.sv`	`_imp_decl(_tlp/_nvme)` ; spec model of the transaction layer; in-order compare
`pcie_nvme_coverage.sv`	Fmt × target × length × byte enables
`pcie_nvme_vseqr.sv` , `pcie_nvme_vseq.sv`	Virtual sequencer (two handles) and virtual sequence (background ready + TLPs)
`pcie_nvme_env.sv`	Both agents, scoreboard, coverage, virtual sequencer
`pcie_nvme_tests.sv`	`pcie_nvme_tests` , `pcie_nvme_err_test` (factory override)
`run.sh`	Compile once, run both tests, logs and waves into `results/<test>/`

8. The 10 constraints

#	Name	Rule	Why
1	`c_type`	Fmt `010` / `011` , Type `00000`	legal Memory Writes
2	`c_fmt_addr`	4-DW header ⇔ address ≥ 4 GB; address DW aligned	spec requester rule

#	Name	Rule	Why
3	`c_target`	window 80%, doorbell 20%	both destinations
4	`c_window`	inside the window and the last byte in the same 4 KB page	spec 4 KB rule — *see the lesson below*
5	`c_doorbell`	doorbell ⇒ Length 1, First BE `F`, one of the 8 doorbell registers	doorbells are 32-bit writes
6	`c_len`	Length 1–32 DW (128 bytes), 1–3 idle cycles	Max Payload Size every device supports
7	`c_payload`	`payload.size() == length`	payload follows the Length field
8	`c_be`	Length 1 ⇒ Last BE 0, First BE ≠ 0; longer ⇒ contiguous BEs	spec byte-enable rules (we keep them contiguous; the spec allows holes only for ≤ 2 DW)
9	`c_good` / err item	no error knob / exactly one	clean traffic, or one fault at a time
10	`c_ready` (`nvme_item`)	ready 70 / 30, held 1–4 cycles	back-pressure from the NVMe side

The lesson behind constraint 4: the first version said `addr + length*4 <= WIN_BASE + 4096`. In 64-bit arithmetic that sum can **wrap around**, so the solver picked addresses like `FFFF_FFFF_FFFF_FFBC` that "obey" the rule — 23 of 150 "good" TLPs were really illegal. The scoreboard agreed with the DUT, so nothing failed: only looking at the counts (errors in a test that should have none) found it. The fix states the window range directly and checks the 4 KB rule on the low 12 bits only.

9. The 10 assertions

#	Where	Name	Rule
1	`tlp_if`	`a_hold`	a beat waiting for `tlp_ready` doesn't change
2	`tlp_if`	`a_sop_v`	`sop` / `eop` only on a valid beat
3	`tlp_if`	`a_frame`	no new `sop` inside a TLP
4	`tlp_if`	`a_no_x`	no X on a valid beat
5	`nvme_if`	`a_hold`	a write waiting for `nvme_ready` doesn't change
6	`nvme_if`	`a_align`	every write is DW aligned
7	`nvme_if`	`a_be`	at least one byte enabled
8	`nvme_if`	`a_db`	doorbell is a one-cycle pulse
9	`nvme_if`	`a_err`	error report: one cycle, non-zero code
10	`nvme_if`	`a_reset`	nothing active right after reset

10. Results and how to run

Test	Result
`pcie_nvme_tests`	150 TLPs: **128 data writes + 22 doorbells** handed off exactly as predicted, 0 errors, 0 assertion failures
`pcie_nvme_err_test`	150 TLPs, **150 errors** reported with the right code
Bug injected (Last BE ignored + doorbell SQ/CQ bit taken from the wrong address bit)	**61 of 150** failed, e.g. `expected WRITE … be e/7 got … be e/f`, `expected DOORBELL SQ3 got CQ3`

Run: `./run.sh` (both tests) or `./run.sh pcie_nvme_err_test`. Logs and `dump.vcd` in `results/<test>/`; waves: `surfer results/pcie_nvme_tests/dump.vcd`. Tools: `../ADDER/setup_tools.sh` once.